NAME : SANDESH NAKADE

CLASS: CSAI-D

ROLL NO: 34

PRN : 12412601

PROBLEM STATEMENT : Perform the following operations using Python on the data sets:

Compute and display summary statistics for each feature available in the dataset. e.g. minimum value, maximum value, mean, range, standard deviation, variance and percentiles Illustrate the feature distributions using histogram. Data cleaning, Data integration, Data transformation, Data model building (e.g. Classification)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


df_csv = pd.read_csv("Exam_Score_Prediction.csv")
```

```
#VIEW OF THE DATA FRAME
df_csv
```

	student_id	age	gender	course	study_hours	class_attendance	internet_access	sleep_hours	sleep_quality	study_me
0	1	17	male	diploma	2.78	92.9	yes	7.4	poor	coa
1	2	23	other	bca	3.37	64.8	yes	4.6	average	online vi
2	3	22	male	b.sc	7.88	76.8	yes	8.5	poor	coa
3	4	20	other	diploma	0.67	48.4	yes	5.8	average	online vi
4	5	20	female	diploma	0.89	71.6	yes	9.8	poor	coa
...	...	...	...	...	...	...	...	...	...	
19995	19997	18	other	bba	6.50	71.3	yes	5.0	good	self-
19996	19998	18	male	b.com	3.71	41.6	no	5.9	average	coa
19997	19999	19	other	diploma	7.88	68.2	yes	4.6	poor	group
19998	20000	19	male	bba	4.60	76.3	no	6.1	good	self-
19999	20001	20	male	b.sc	7.50	47.9	yes	7.5	poor	coa

20000 rows × 13 columns

```
a = df_csv.describe();
a
```

Show hidden output

```
df_csv["exam_score"].min()
```

```
19.599
```

```
df_csv["exam_score"].max()
```

```
100.0
```

```
#RANGE OF THE ATTRIBUTE WHICH ARE NUMERIC
df_csv.max(numeric_only=True) - df_csv.min(numeric_only=True)
```

Show hidden output

```
df_csv["exam_score"].mean()
```

```
np.float64(62.513225)
```

```
df_csv["exam_score"].var()
```

```
357.53103816127714
```

```
df_csv["exam_score"].std()
```

```
18.908491165645056
```

```
#variance
df_csv.var(numeric_only=True)
```

	0
student_id	3.333509e+07
age	5.218748e+00
study_hours	5.328308e+00
class_attendance	2.986766e+02
sleep_hours	3.000135e+00
exam_score	3.575310e+02

dtype: float64

Double-click (or enter) to edit

```
#USER DEFINED FUNCTION
def mean_score(df_csv):
  total = 0
  count =0
  for score in df_csv["exam_score"]:
      count+=1
      total += score

  return total/count

def min_score(df_csv):
  min = 101
  for score in df_csv["exam_score"]:
    if score < min:
      min = score
  return min

def max_score(df_csv):
  max = 0;
  for score in df_csv["exam_score"]:
    if score > max:
      max = score
  return max

import math

def standard_deviation(df_csv):
    mean = mean_score(df_csv)
    total = 0
    n = len(df_csv)

    for score in df_csv["exam_score"]:
        total += (score - mean) ** 2

    variance = total / n
    std_dev = math.sqrt(variance)

    return std_dev


print(max_score(df_csv))
print(min_score(df_csv))
print(mean_score(df_csv))
print(standard_deviation(df_csv))
```

```
100.0
19.599
62.51322500000027
18.90801844745686
```

```
Q1 = df_csv["exam_score"].quantile(0.25)
```

```
Q2 = df_csv["exam_score"].quantile(0.5)
Q3 = df_csv["exam_score"].quantile(0.75)

print("Q1 : ",Q1,"Q2 : ",Q2,"Q3 : ",Q3)

q1 = np.percentile(df_csv["exam_score"],25);

q2 = np.percentile(df_csv["exam_score"],50)
q3 = np.percentile(df_csv["exam_score"],75)

print("q1 : ",q1,"q2 : ",q2,"q3 : ",q3)
```

```
Q1 :  48.8 Q2 :  62.6 Q3 :  76.3
q1 :  48.8 q2 :  62.6 q3 :  76.3
```

```
#OUTLIERS USING QUARTILE
Q1 = df_csv["exam_score"].quantile(0.25)
Q2 = df_csv["exam_score"].quantile(0.5)
Q3 = df_csv["exam_score"].quantile(0.75)

IQR  = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

print("LOWER : ",lower)
print("UPPER : ",upper)

outliers = df_csv[(df_csv["exam_score"]<lower) | (df_csv["exam_score"]>upper)]

print(outliers["exam_score"])
```

```
LOWER :  7.549999999999997
UPPER :  117.55
Series([], Name: exam_score, dtype: float64)
```

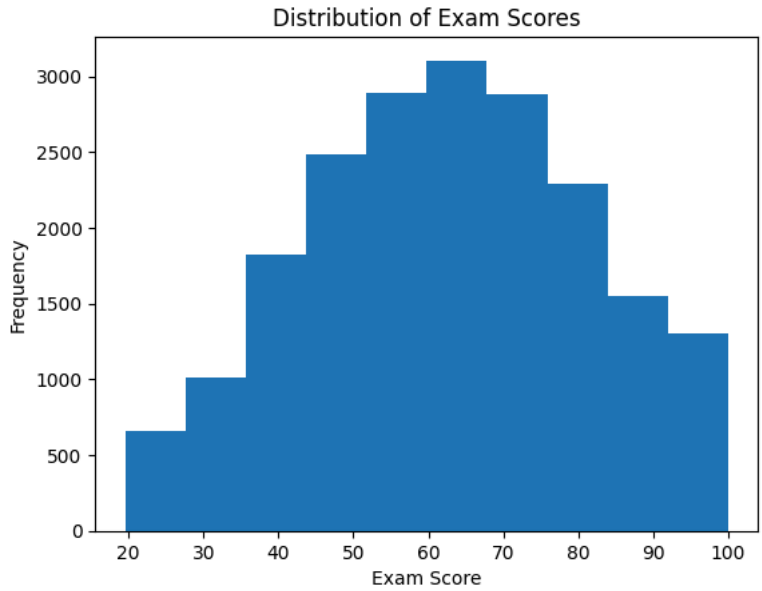
```
#drawing histogram for exam score

plt.hist(df_csv["exam_score"], bins=10)

plt.xlabel("Exam Score")

plt.ylabel("Frequency")

plt.title("Distribution of Exam Scores")
plt.show()
```




```
data = [
    [1, "Amit",   "Male",   20, "SE", 101, "Computer Science", 8.2, "Pune"],
    [2, "Riya",   "Female", 19, "SE", 102, "Computer Science", 8.5, "Mumbai"],
    [3, "Rohan",  "Male",   21, "TE", 103, "Information Technology", 7.8, "Pune"],
    [4, "Sneha",  "Female", 20, "SE", 104, "Computer Science", np.nan, "Delhi"],
    [1, "Amit",   "Male",   20, "SE", 101, "Computer Science", 8.2, "Pune"],   # duplicate
```

```
    [6, "Kunal",  "M",      22, "BE", 105, "Mechanical", 6.9, "Nagpur"],
    [7, "Anita",  "Female", np.nan, "TE", 106, "Information Technology", 7.5, "Bangalore"],
    [8, Rahul",  "male",   21, "TE", 107, "Computer Science", 8.0, np.nan],
    [9, Neha",   "F",      19, "SE", 108, "Computer Science", 9.1, "Mumbai"],
    [10,"Arjun",  "Male",   23, "BE", 109, "Civil", 6.5, "Chennai"],
    [11,"Priya",  "Female", 20, "SE", 110, "Computer Science", 8.7, "Pune"],
    [12,"Sameer", "Male",   21, "TE", 111, "Information Technology", np.nan, "Pune"],
    [13,"Kiran",  np.nan,   22, "BE", 112, "Mechanical", 7.2, "Nagpur"],
    [2,"Riya",   "Female", 19, "SE", 102, "Computer Science", 8.5, "Mumbai"], # duplicate
    [15,"Manish", "Male",   20, "SE", 113, "Computer Science", 8.0, "Delhi"]
]

columns = [
    "id", "name", "gender", "age", "class",
    "roll_no", "course", "cgpa", "address"
]

df = pd.DataFrame(data, columns=columns)

df
```

	id	name	gender	age	class	roll_no	course	cgpa	address
0	1	Amit	Male	20.0	SE	101	Computer Science	8.2	Pune
1	2	Riya	Female	19.0	SE	102	Computer Science	8.5	Mumbai
2	3	Rohan	Male	21.0	TE	103	Information Technology	7.8	Pune
3	4	Sneha	Female	20.0	SE	104	Computer Science	NaN	Delhi
4	1	Amit	Male	20.0	SE	101	Computer Science	8.2	Pune
5	6	Kunal	M	22.0	BE	105	Mechanical	6.9	Nagpur
6	7	Anita	Female	NaN	TE	106	Information Technology	7.5	Bangalore
7	8	Rahul	male	21.0	TE	107	Computer Science	8.0	NaN
8	9	Neha	F	19.0	SE	108	Computer Science	9.1	Mumbai
9	10	Arjun	Male	23.0	BE	109	Civil	6.5	Chennai
10	11	Priya	Female	20.0	SE	110	Computer Science	8.7	Pune
11	12	Sameer	Male	21.0	TE	111	Information Technology	NaN	Pune
12	13	Kiran	NaN	22.0	BE	112	Mechanical	7.2	Nagpur
13	2	Riya	Female	19.0	SE	102	Computer Science	8.5	Mumbai
14	15	Manish	Male	20.0	SE	113	Computer Science	8.0	Delhi

```
df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   id       15 non-null     int64
 1   name     15 non-null     object
 2   gender   14 non-null     object
 3   age      14 non-null     float64
 4   class    15 non-null     object
 5   roll_no  15 non-null     int64
 6   course   15 non-null     object
 7   cgpa     13 non-null     float64
 8   address  14 non-null     object
dtypes: float64(2), int64(2), object(5)
memory usage: 1.2+ KB
```

	id	age	roll_no	cgpa
count	15.000000	14.000000	15.000000	13.000000
mean	6.933333	20.500000	106.266667	7.930769
std	4.651677	1.224745	4.096456	0.742052
min	1.000000	19.000000	101.000000	6.500000
25%	2.500000	20.000000	102.500000	7.500000
50%	7.000000	20.000000	106.000000	8.000000
75%	10.500000	21.000000	109.500000	8.500000
max	15.000000	23.000000	113.000000	9.100000

```
df.duplicated().sum()
df[df.duplicated()]
```

	id	name	gender	age	class	roll_no	course	cgpa	address
4	1	Amit	Male	20.0	SE	101	Computer Science	8.2	Pune
13	2	Riya	Female	19.0	SE	102	Computer Science	8.5	Mumbai

```
#missing values
df["age"].fillna(df["age"].median(), inplace=True)
df["gender"].fillna("Unknown", inplace=True)


df["gender"] = df["gender"].str.lower()

df["gender"] = df["gender"].replace({
    "m": "male",
    "f": "female"
})


df
```

```
/tmp/ipython-input-1395743376.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df["age"].fillna(df["age"].median(), inplace=True)
/tmp/ipython-input-1395743376.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df["gender"].fillna("Unknown", inplace=True)
```

	id	name	gender	age	class	roll_no	course	cgpa	address
0	1	Amit	male	20.0	SE	101	Computer Science	8.200000	Pune
1	2	Riya	female	19.0	SE	102	Computer Science	8.500000	Mumbai
2	3	Rohan	male	21.0	TE	103	Information Technology	7.800000	Pune
3	4	Sneha	female	20.0	SE	104	Computer Science	7.930769	Delhi
4	1	Amit	male	20.0	SE	101	Computer Science	8.200000	Pune
5	6	Kunal	male	22.0	BE	105	Mechanical	6.900000	Nagpur
6	7	Anita	female	20.0	TE	106	Information Technology	7.500000	Bangalore
7	8	Rahul	male	21.0	TE	107	Computer Science	8.000000	Unknown
8	9	Neha	female	19.0	SE	108	Computer Science	9.100000	Mumbai
9	10	Arjun	male	23.0	BE	109	Civil	6.500000	Chennai
10	11	Priya	female	20.0	SE	110	Computer Science	8.700000	Pune
11	12	Sameer	male	21.0	TE	111	Information Technology	7.930769	Pune
12	13	Kiran	unknown	22.0	BE	112	Mechanical	7.200000	Nagpur
13	2	Riya	female	19.0	SE	102	Computer Science	8.500000	Mumbai
14	15	Manish	male	20.0	SE	113	Computer Science	8.000000	Delhi

```
Q1 = df["cgpa"].quantile(0.25)
Q3 = df["cgpa"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df["cgpa"] = df["cgpa"].clip(lower, upper)
df["cgpa"]
```

	cgpa
0	8.200000
1	8.500000
2	7.800000
3	7.930769
4	8.200000
5	6.900000
6	7.500000
7	8.000000
8	9.100000
9	6.600000
10	8.700000
11	7.930769
12	7.200000
13	8.500000
14	8.000000

dtype: float64

```
#concatenate function
jan = pd.DataFrame ({
    'id' : [1,2,3],
    'sales' : [200,300,400]
})

feb = pd.DataFrame({
    'id' : [1,5,6],
    'sales' : [100,300,400]
})

sales_data = pd.concat([jan,feb],ignore_index=True)
sales_data
```

	id	sales
0	1	200
1	2	300
2	3	400
3	1	100
4	5	300
5	6	400

```
#row wise concatenation
jan = pd.DataFrame ({
    'id' : [1,2,3],
    'sales' : [200,300,400]
})

feb = pd.DataFrame({
    'id' : [1,5,6],
    'sales' : [100,300,400]
})

sales_data = pd.concat
```

```
#joins
students = pd.DataFrame({
    'id' : [1,2,3,4],
    'name' : ["amit","riya","rohan","sneha"]
})

results = pd.DataFrame({
    'id' : [1,2,4],
    'marks' : [90,89,45]
})



mer_data_outer  = pd.merge(students,results,on='id',how='outer')
mer_data_outer
```

	id	name	marks
0	1	amit	90
1	2	riya	89
2	4	sneha	45

```
mer_data_inner  = pd.merge(students,results,on='id',how='inner')
mer_data_inner
```

	id	name	marks
0	1	amit	90
1	2	riya	89
2	4	sneha	45

```
mer_left = pd.merge(students,results,on='id',how='left')
mer_left
```

	id	name	marks
0	1	amit	90.0
1	2	riya	89.0
2	3	rohan	NaN